

Practical: 7

Implementation of making a change problem using dynamic programming

/08/2026

Aim:

Implementation of making a change problem using dynamic programming

Algorithm:

Algorithm:-

Input:- No. of coins denominations n , coin array $coins[]$ and target amount A

Output:- minimum and maximum no. of coins required to make amount A

Step1:- Start

Step2:- Read the number of coins n

Step3:- Read the n coin denominations into $coins[]$

Step4:- Read the target amount A

Step5:- create two dp arrays
 $dp_min[0..A]$
 $dp_max[0..A]$

Step6:- initialize $dp_min[0]=0$
 $dp_max[0]=0$

Step7:- for every amount from 1 to A check every coin $coins[i]$

Step8:- for every amount from 1 to A initialize
 $dp_min[i]=\infty$
 $dp_max[i]=\infty$

Step9:- if $coins[i] \leq i$

Step10:- calculate the minimum no. of coins
 $dp_min[i] = \min(dp_min[i], dp_min[i - coins[i]] + 1)$
 calculate the maximum no. of coins
 $dp_max[i] = \max(dp_max[i], dp_max[i - coins[i]] + 1)$

Step11:- Repeat Step 8 - 10 until all amount from 1 to A are processed

Step12:- minimum no. of coins is $dp_min[A]$

Step13:- maximum no. of coins is $dp_max[A]$

Step14:- Stop

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

class CoinChange {
private:
    int coins[10];
    int n;
    int amount;

    int dpMin[100];
    int dpMax[100];

public:
    void getInput() {
        cout << "Enter number of coins: ";
        cin >> n;

        cout << "Enter coin denominations: ";
        for (int i = 0; i < n; i++) {
            cin >> coins[i];
        }

        cout << "Enter amount: ";
        cin >> amount;
    }

    void solve() {

        for (int i = 0; i <= amount; i++) {
            dpMin[i] = amount + 1;
```

```
    dpMax[i] = -1;
}

dpMin[0] = 0;
dpMax[0] = 0;

for (int i = 1; i <= amount; i++) {

    for (int j = 0; j < n; j++) {

        if (coins[j] <= i) {

            dpMin[i] = min(dpMin[i],
                           dpMin[i - coins[j]] + 1);

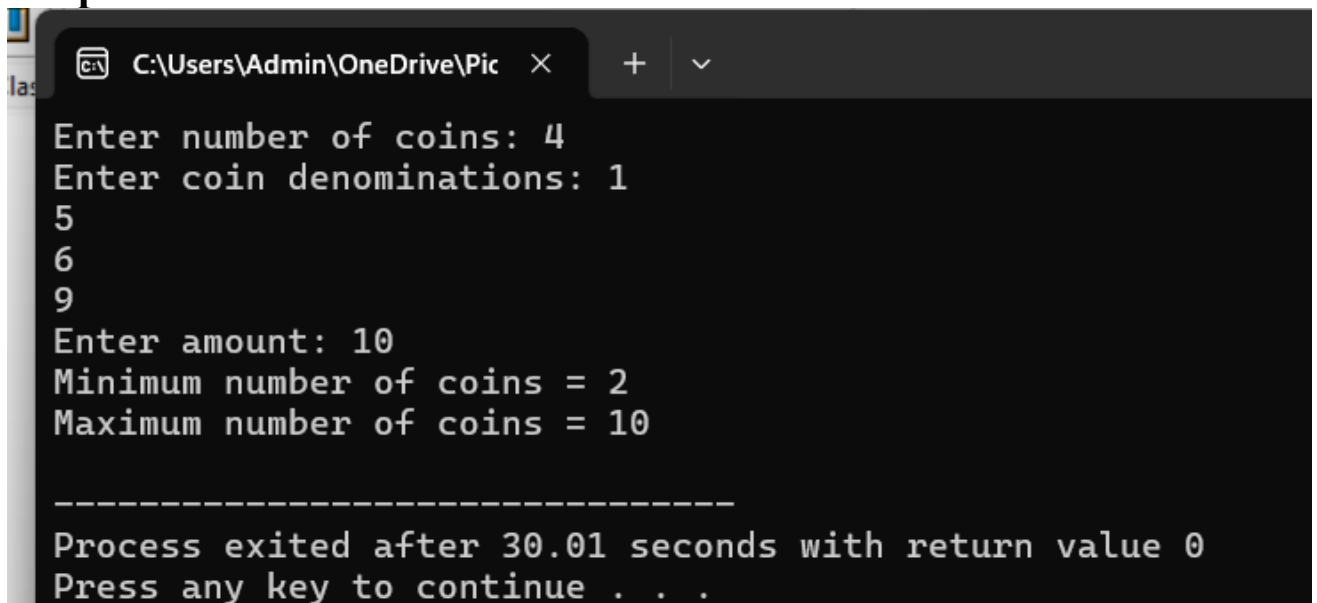
            if (dpMax[i - coins[j]] != -1) {
                dpMax[i] = max(dpMax[i],
                               dpMax[i - coins[j]] + 1);
            }
        }
    }
}

if (dpMin[amount] > amount) {
    cout << "Change cannot be made." << endl;
}
else {
    cout << "Minimum number of coins = "
         << dpMin[amount] << endl;

    cout << "Maximum number of coins = "
         << dpMax[amount] << endl;
}
```

```
}  
};  
  
int main() {  
  
    CoinChange c;  
  
    c.getInput();  
    c.solve();  
  
    return 0;  
}
```

Output:



```
C:\Users\Admin\OneDrive\Pic x + v  
Enter number of coins: 4  
Enter coin denominations: 1  
5  
6  
9  
Enter amount: 10  
Minimum number of coins = 2  
Maximum number of coins = 10  
  
-----  
Process exited after 30.01 seconds with return value 0  
Press any key to continue . . .
```

Time Complexity:

Time complexity:-

Let n = number of coin denominations
 A = target amount
solve function
the program contain two nested loops

```
for (int i = 1; i <= amount; i++) {  
    for (int j = 0; j < n; j++) {
```

* The outer loop executed A times
→ The inner loop every amount, all n coins are checked

$$T(n, A) = O(A) \times O(n)$$
$$= O(nA)$$

Time complexity = $T(n, A) = O(nA)$

Best case: event if a solution is found immediately the loop continue

$$T(n) = O(nA)$$

Average case: - On an average, the same dp table is filled and all coins are considered

$$T(n) = O(nA)$$

Worst case: - All amount and all coin denominations are processed

$$T(n) = O(nA)$$

Conclusion:

The Making Change Problem was successfully implemented using Dynamic Programming to determine both the minimum and maximum number of coins required to make a given amount. The method efficiently solves the problem by storing solutions to smaller subproblems and avoids repeated calculations.